Project Title

MATRIX CHAIN MULTIPLICATION OPTIMIZATION USING INTERVAL DYNAMIC PROGRAMMING

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by

K. Girish – 2520030231

P. Venkat – 2520030450

N.V. Karthkeya – 2520030307

Team Number: 9

Section: 7

Under the guidance of

Dr. S. Madhavi

Assistant Professor, CSE

Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



DECLARATION

The Mini Project Report entitled “Matrix Chain Multiplication Optimization Using Interval Dynamic Programming” is a record of bonafide work by K. Girish – 2520030231, P. Venkat – 2520030450, and N.V. Karthkeya – 2520030307, submitted in partial fulfillment of the requirements for the award of B.Tech in Computer Science and Engineering at K L (Deemed to be) University. The results embodied in this report have not been copied from any other department, university, or institute.

K. Girish – 2520030231,

P. Venkat – 2520030450,
N.V. Karthkeya – 2520030307.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project-based report entitled “Matrix Chain Multiplication Optimization Using Interval Dynamic Programming” is bonafide work done and submitted by K. Girish – 2520030231, P. Venkat – 2520030450, and N.V. Karthkeya – 2520030307, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at K L (Deemed to be) University during the academic year 2026–2027.

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my guide, Dr. S. Madhavi, Assistant Professor, CSE which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P. Venkateshwar Rao (CSE)** Head of the *Department for Computer Science Engineering* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

K. Girish – 2520030231

P. Venkat – 2520030450

N.V. Karthkeya – 2520030307

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	16
8.	Conclusion & Future Scope	18
9.	References	19

1.ABSTRACT

Matrix Chain Optimizer 3D is a Java-based web application that demonstrates matrix-chain multiplication optimization using interval dynamic programming. Users can provide an image or a CSV dataset as input. An image is converted into a compact grayscale numeric matrix, while numeric columns from a CSV dataset are used to construct the working matrix. The input drives the algorithmic analysis and does not require manually entered class labels.

The application displays the input matrix and its dimensions, then demonstrates algorithmic views derived from the same input. These include sequence alignment and string-pattern matching, matrix-chain cost optimization, additional dynamic-programming demonstrations, and a capacity network solved with the Edmonds–Karp maximum-flow algorithm. The interface is built with HTML, CSS, and JavaScript and is served by a Java 17 Maven application.

The project makes algorithmic behavior visible through input-derived matrices, tables, sequence views, optimal parenthesization, minimum multiplication cost, and maximum-flow steps. It provides an interactive way to explore how several Data Structures and Algorithms techniques can operate on data derived from a single image or numeric CSV dataset.

Team Members:

1. K. Girish – 2520030231
2. P. Venkat – 2520030450
3. 1. K. Girish – 2520030231

2. P. Venkat – 2520030450

3. N.V. Karthkeya – 2520030307

2. INTRODUCTION

The Matrix Chain Optimizer 3D application is a software-based solution designed to simplify and automate the process of assigning students to laboratory rooms. In educational institutions, allocating a large number of students to different laboratories manually can be time-consuming and may lead to uneven distribution, capacity violations, and difficulties in managing student records.

The application accepts either an image or a CSV dataset. Image pixels are converted to grayscale numeric values and sampled when needed to keep the matrix view readable. For CSV input, numeric feature columns are used and non-numeric columns are ignored. The resulting matrix feeds the project's algorithmic views.

The main objective is to make algorithmic analysis interactive and understandable by deriving matrix data from user-provided inputs and presenting the results in tables and visual views. Users can inspect the input matrix, compare matrix multiplication costs, view pattern-matching results, and examine the maximum-flow calculation.

The project focuses on a single-page interface in which image or CSV input drives the analysis. The matrix preview, derived sequences, dynamic-programming tables, parenthesization, and network-flow steps are intended to reflect the selected input instead of relying on fixed demonstration outputs.

The remainder of this document describes the problem statement, objectives, existing and proposed systems, system requirements, methodology, system design, implementation, results, and future enhancements of the proposed Matrix Chain Optimizer 3D.

3. LITERATURE SURVEY

3.1 Matrix Chain Multiplication

University timetabling systems are designed to allocate students, faculty, classrooms, and laboratory resources according to predefined constraints. These systems help institutions reduce scheduling conflicts and make efficient use of available resources. Many solutions use constraint-based approaches to handle room capacity, student groups, subjects, and available time slots. The study of such systems provides a foundation for developing an automated laboratory allocation system that can distribute students efficiently among available laboratories.

3.2 Interval Dynamic Programming

Automated classroom allocation systems focus on assigning classes or student groups to available rooms based on room capacity and other constraints. These systems reduce the effort required for manual allocation and help prevent situations where the number of students exceeds the available seating capacity. Room allocation approaches can be extended to laboratory allocation by considering the capacity and physical arrangement of each laboratory. Such systems are useful for designing an efficient resource allocation mechanism.

3.3 String Pattern Matching

Examination seating arrangement systems automatically assign students to examination rooms according to available seats and institutional rules. These systems generally use student information and room capacity as input and generate an organized seating arrangement. They help minimize manual work and reduce errors during large-scale examinations. The concepts used in examination seating allocation are closely related to the proposed project because both systems require students to be distributed among available rooms without exceeding capacity.

3.4 Sequence Alignment

Course scheduling systems manage the allocation of courses to classrooms and laboratories while considering room availability and capacity. Such systems are particularly useful in educational institutions where multiple departments share limited laboratory resources. Scheduling algorithms can improve resource utilization and avoid conflicts between different student groups. These approaches provide useful ideas for developing a system that assigns students to different laboratories based on their available capacity.

3.5 Dynamic Programming Demonstrations

Constraint-based scheduling is widely used for solving complex allocation problems. In educational scheduling, constraints may include room capacity, availability, student groups, faculty requirements, and resource limitations. The system searches for an allocation that satisfies the required constraints while making efficient use of available resources. This approach is relevant to the proposed Matrix Chain Optimizer 3D because student allocation must respect the capacity constraints of each laboratory.

3.6 Network Flow and Maximum Flow

Genetic algorithms have been applied to university timetabling and resource allocation problems. These algorithms generate possible solutions and iteratively improve them according to a fitness function. They are useful when the number of students, rooms, subjects, and constraints becomes large. Although the proposed project can use a simpler allocation strategy, studying genetic algorithm-based scheduling helps understand how optimization techniques can be applied to improve resource utilization and allocation quality.

3.7 Image and CSV Numeric Data Processing

Student allocation systems are developed to distribute students among available classes, sections, rooms, or other academic resources. These systems generally process student records and use predefined rules to generate allocations. Automated allocation reduces repetitive administrative work and makes the allocation process more consistent. The proposed project follows a similar concept by taking a student dataset and a laboratory dataset and generating an organized laboratory assignment.

3.8 Interactive Algorithm Visualization

Modern resource scheduling platforms provide automated methods for assigning limited resources to multiple users or groups. They commonly consider resource availability, capacity, and user requirements before generating an allocation. These concepts can be applied to academic laboratory management, where laboratories are limited resources and students need to be distributed efficiently. The proposed project adapts these resource-allocation principles specifically for student laboratory allocation.

4. Problem Statement and Objectives

Problem Statement

Matrix-chain multiplication can require many scalar operations, and the cost depends on the order in which intermediate products are formed. A user-friendly tool is needed to accept data-derived matrix input, calculate an efficient parenthesization, and demonstrate related algorithms using the same input.

Problem Description

Without a clear visualization, it can be difficult to understand how interval dynamic programming fills the cost table and selects the optimal split points. The project provides an interactive application that converts an image or CSV input into a numeric matrix and presents the algorithmic results derived from that matrix.

Objectives:

- To accept an image or numeric CSV dataset as input.
- To convert image pixels or numeric CSV columns into a working matrix.
- To display the dimensions and a readable preview of the input matrix.
- To calculate matrix-chain multiplication cost using interval dynamic programming.
- To derive and display the optimal matrix parenthesization.
- To demonstrate string pattern matching and sequence alignment on input-derived sequences.
- To demonstrate an input-derived capacity network using Edmonds–Karp maximum flow.
- To provide a clear interactive interface for exploring algorithm outputs.
- To support different image and numeric CSV inputs.
- To keep the displayed algorithmic results driven by the selected input.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: Intel Core i3 or above
- RAM: Minimum 4 GB
- Storage: Minimum 500 MB free space

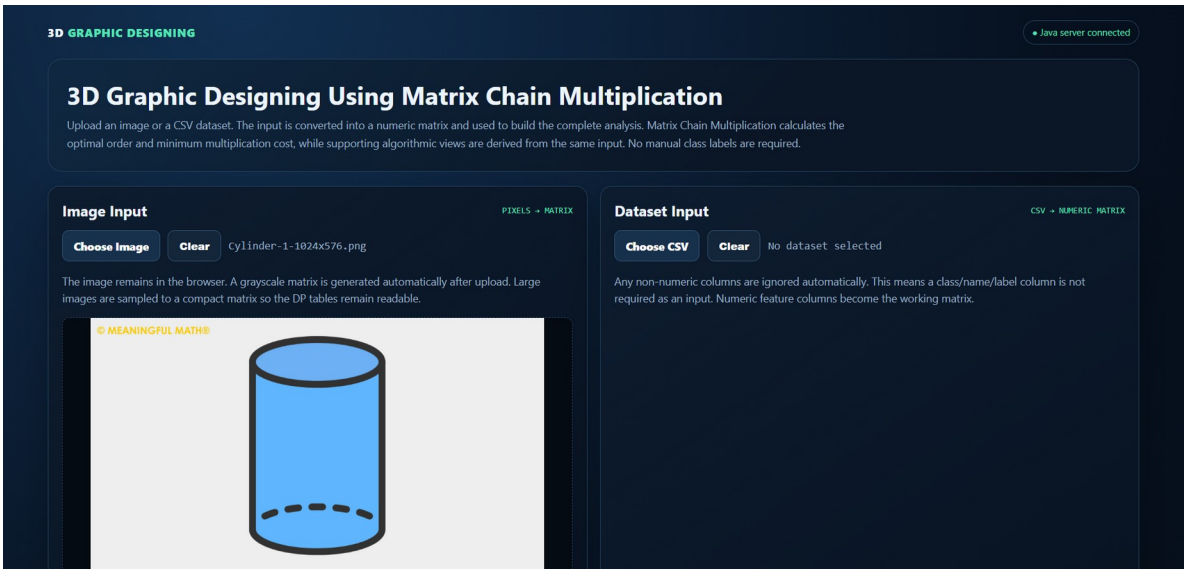
Software Requirements:

- Operating System: Windows / Linux
- Frontend: HTML, CSS, JavaScript
- Backend: Java 17

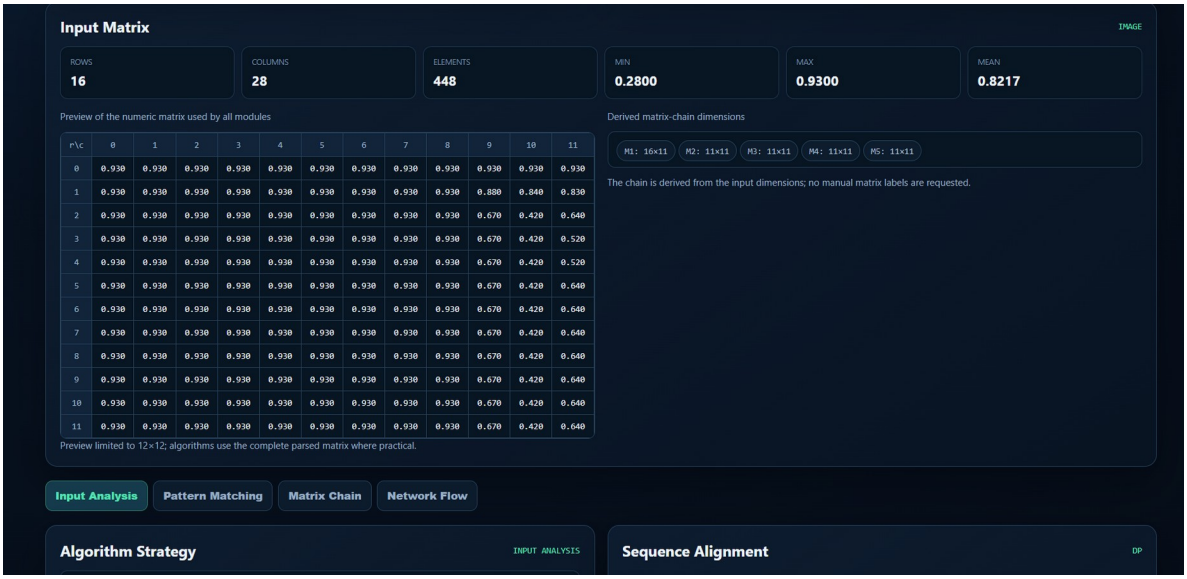
- Build Tool: Apache Maven
- Input Formats: Image and CSV dataset
- Web Browser: Google Chrome / Microsoft Edge

6. IMPLEMENTATION

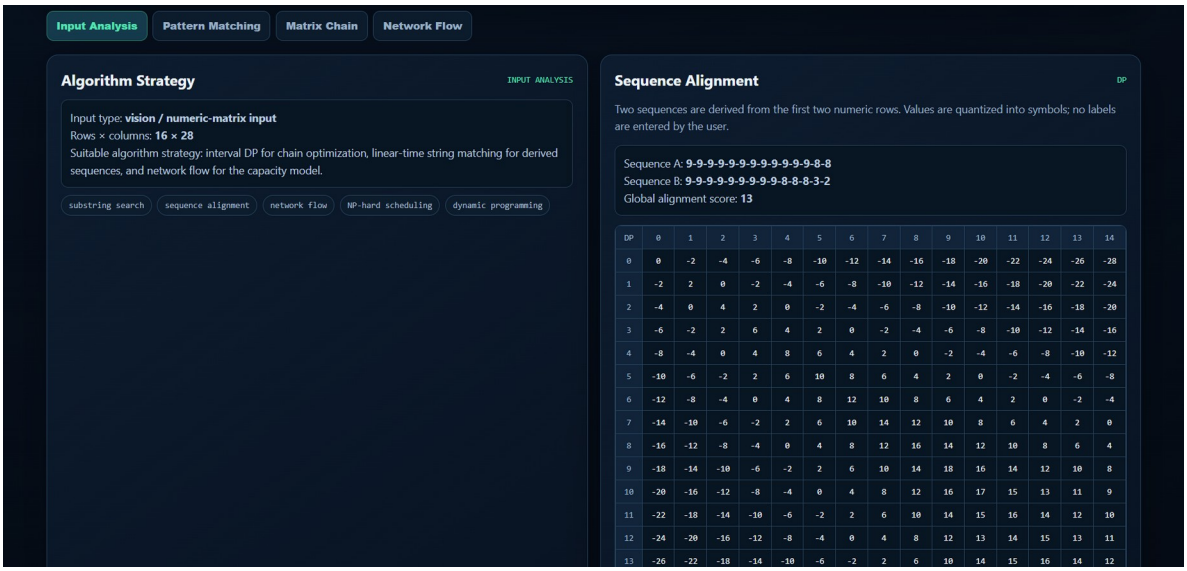
Module 1: Input Matrix and Input Analysis



Module 2: String Pattern Matching and Sequence Alignment

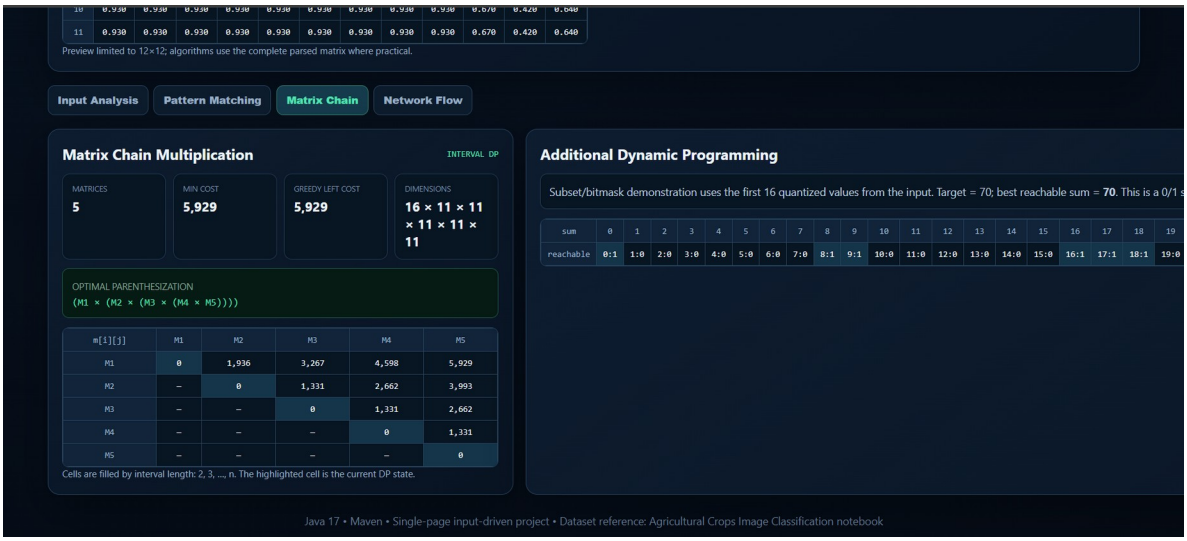


Module 3: Matrix Chain Multiplication using Interval DP



DP	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
0	0	-2	-4	-6	-8	-10	-12	-14	-16	-18	-20	-22	-24	-26	-28
1	-2	2	0	-2	-4	-6	-8	-10	-12	-14	-16	-18	-20	-22	-24
2	-4	0	4	2	0	-2	-4	-6	-8	-10	-12	-14	-16	-18	-20
3	-6	-2	2	6	4	2	0	-2	-4	-6	-8	-10	-12	-14	-16
4	-8	-4	0	4	8	6	4	2	0	-2	-4	-6	-8	-10	-12
5	-10	-6	-2	2	6	10	8	6	4	2	0	-2	-4	-6	-8
6	-12	-8	-4	0	4	8	12	10	8	6	4	2	0	-2	-4
7	-14	-10	-6	-2	2	6	10	14	12	10	8	6	4	2	0
8	-16	-12	-8	-4	0	4	8	12	16	14	12	10	8	6	4
9	-18	-14	-10	-6	-2	2	6	10	14	18	16	14	12	10	8
10	-20	-16	-12	-8	-4	0	4	8	12	16	17	15	13	11	9
11	-22	-18	-14	-10	-6	-2	2	6	10	14	15	16	14	12	10
12	-24	-20	-16	-12	-8	-4	0	4	8	12	13	14	15	13	11
13	-26	-22	-18	-14	-10	-6	-2	2	6	10	14	15	16	14	12

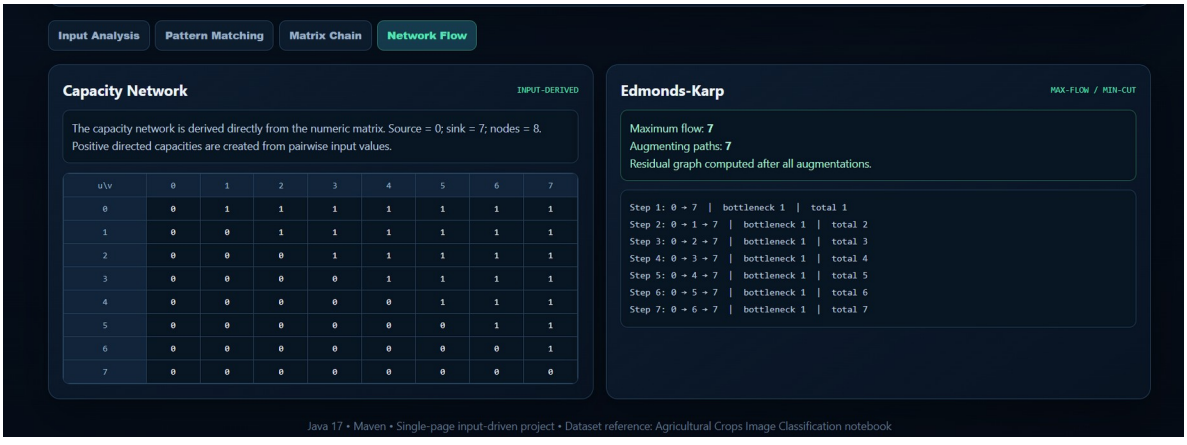
Module 4: Capacity Network and Edmonds–Karp Maximum Flow



m[i][j]	M1	M2	M3	M4	M5
M1	0	1,936	3,267	4,598	5,929
M2	–	0	1,331	2,662	3,993
M3	–	–	0	1,331	2,662
M4	–	–	–	0	1,331
M5	–	–	–	–	0

sum	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
reachable	0:1	1:0	2:0	3:0	4:0	5:0	6:0	7:0	8:1	9:1	10:0	11:0	12:0	13:0	14:0	15:0	16:1	17:1	18:1	19:0

7. RESULTS & DISCUSSION



u\v	0	1	2	3	4	5	6	7
0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	1
2	0	0	0	1	1	1	1	1
3	0	0	0	0	1	1	1	1
4	0	0	0	0	0	1	1	1
5	0	0	0	0	0	0	1	1
6	0	0	0	0	0	0	0	1
7	0	0	0	0	0	0	0	0

String Pattern Matching

KMP • Z • RABIN-KARP

A deterministic symbol string is generated from the input matrix. Choose a method and a pattern length to compare the three linear-time approaches.

METHOD: KMP

PATTERN LENGTH: 3

Generated sequence length: **448**

Pattern: **jjj**

KMP matches: **0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255, 256, 257, 258, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 435, 436, 437, 438, 439, 440, 441, 442, 443, 444, 445**

Suffix Structures

SA • LCP

Suffix array contains 80 suffix positions. LCP[i] stores the common-prefix length of adjacent sorted suffixes.

rank	suffix index	LCP	suffix preview
0	41	0	cceiiijjjjjjjjjjjjjjjjj
1	42	1	ceiiijjjjjjjjjjjjjjjjjg
2	40	0	dcceiiijjjjjjjjjjjjjjjj
3	66	0	egggggggegjjjjjjjjjjjjj
4	73	2	egjjjjjjjjjjjjjjjjjgefe
5	43	1	eiiijjjjjjjjjjjjjjjjjge
6	65	0	gegggggggegjjjjjjjjjjjj
7	72	3	gegjjjjjjjjjjjjjjjjjgef
8	71	1	ggegjjjjjjjjjjjjjjjjjge
9	70	2	gggegjjjjjjjjjjjjjjjjjg
10	69	3	ggggegjjjjjjjjjjjjjjjjj
11	68	4	gggggegjjjjjjjjjjjjjjjj
12	67	5	ggggggegjjjjjjjjjjjjjjj
13	74	1	gjjjjjjjjjjjjjjjjjgefee
14	39	0	idcceiiijjjjjjjjjjjjjjj
15	38	1	iidcceiiijjjjjjjjjjjjjj
16	37	2	iiidcceiiijjjjjjjjjjjjj

3D GRAPHIC DESIGNING

• Java server connected

3D Graphic Designing Using Matrix Chain Multiplication

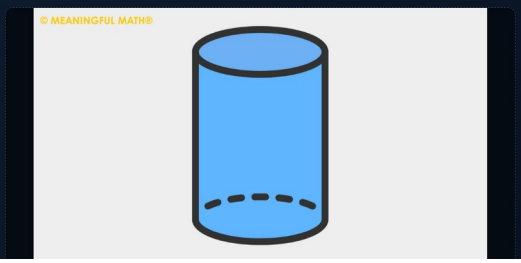
Upload an image or a CSV dataset. The input is converted into a numeric matrix and used to build the complete analysis. Matrix Chain Multiplication calculates the optimal order and minimum multiplication cost, while supporting algorithmic views are derived from the same input. No manual class labels are required.

Image Input

PIXELS → MATRIX

Choose Image | Clear | Cylinder-1-1024x576.png

The image remains in the browser. A grayscale matrix is generated automatically after upload. Large images are sampled to a compact matrix so the DP tables remain readable.



Dataset Input

CSV → NUMERIC MATRIX

Choose CSV | Clear | No dataset selected

Any non-numeric columns are ignored automatically. This means a class/name/label column is not required as an input. Numeric feature columns become the working matrix.

Input Matrix

IMAGE

ROWS	COLUMNS	ELEMENTS	MIN	MAX	MEAN
16	28	448	0.2800	0.9300	0.8217

Preview of the numeric matrix used by all modules

r\c	0	1	2	3	4	5	6	7	8	9	10	11
0	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930
1	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.880	0.840	0.830
2	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.670	0.420	0.640
3	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.670	0.420	0.520
4	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.670	0.420	0.520
5	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.670	0.420	0.640
6	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.670	0.420	0.640
7	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.670	0.420	0.640
8	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.670	0.420	0.640
9	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.670	0.420	0.640
10	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.670	0.420	0.640
11	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.930	0.670	0.420	0.640

Preview limited to 12×12; algorithms use the complete parsed matrix where practical.

Derived matrix-chain dimensions

M1: 16×11 M2: 11×11 M3: 11×11 M4: 11×11 M5: 11×11

The chain is derived from the input dimensions; no manual matrix labels are requested.

Input Analysis | Pattern Matching | Matrix Chain | Network Flow

Algorithm Strategy

INPUT ANALYSIS

Input type: **vision / numeric-matrix input**
Rows × columns: **16 × 28**
Suitable algorithm strategy: interval DP for chain optimization, linear-time string matching for derived sequences, and network flow for the capacity model.

substring search | sequence alignment | network flow | NP-hard scheduling | dynamic programming

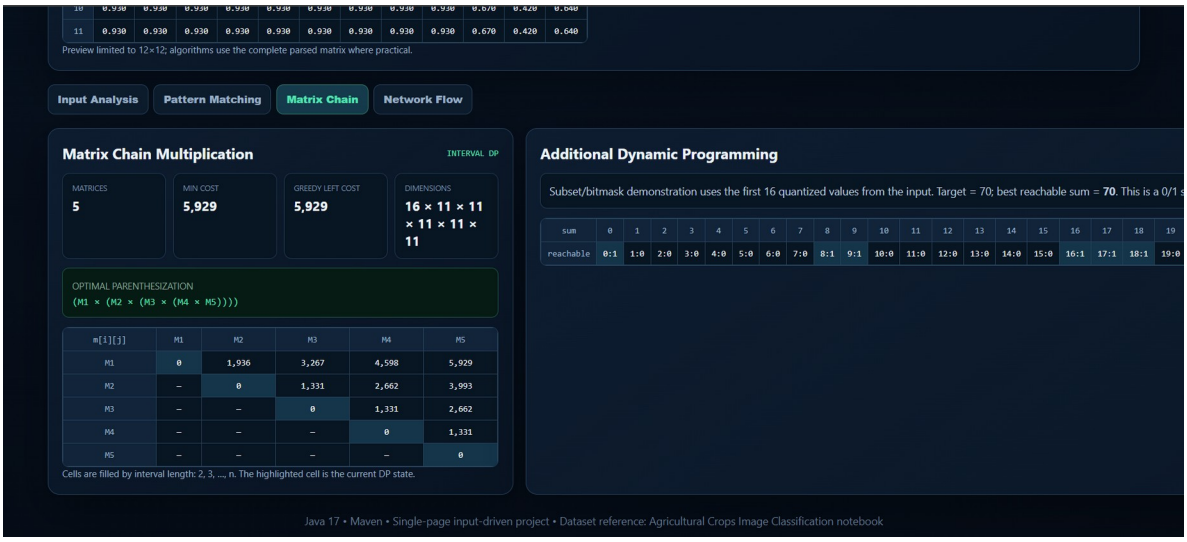
Sequence Alignment

DP

Two sequences are derived from the first two numeric rows. Values are quantized into symbols; no labels are entered by the user.

Sequence A: **9-9-9-9-9-9-9-9-9-9-9-8-8**
Sequence B: **9-9-9-9-9-9-9-9-8-8-8-3-2**
Global alignment score: **13**

DP	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
0	0	-2	-4	-6	-8	-10	-12	-14	-16	-18	-20	-22	-24	-26	-28
1	-2	2	0	-2	-4	-6	-8	-10	-12	-14	-16	-18	-20	-22	-24
2	-4	0	4	2	0	-2	-4	-6	-8	-10	-12	-14	-16	-18	-20
3	-6	-2	2	6	4	2	0	-2	-4	-6	-8	-10	-12	-14	-16
4	-8	-4	0	4	8	6	4	2	0	-2	-4	-6	-8	-10	-12
5	-10	-6	-2	2	6	10	8	6	4	2	0	-2	-4	-6	-8
6	-12	-8	-4	0	4	8	12	10	8	6	4	2	0	-2	-4
7	-14	-10	-6	-2	2	6	10	14	12	10	8	6	4	2	0
8	-16	-12	-8	-4	0	4	8	12	16	14	12	10	8	6	4
9	-18	-14	-10	-6	-2	2	6	10	14	18	16	14	12	10	8
10	-20	-16	-12	-8	-4	0	4	8	12	16	17	15	13	11	9
11	-22	-18	-14	-10	-6	-2	2	6	10	14	15	16	14	12	10
12	-24	-20	-16	-12	-8	-4	0	4	8	12	13	14	15	13	11
13	-26	-22	-18	-14	-10	-6	-2	2	6	10	14	15	16	14	12



8. CONCLUSION & FUTURE SCOPE

Conclusion

The Student–Matrix Chain Optimizer 3D successfully addresses the problem of manually assigning students to multiple laboratories. The system processes student and laboratory datasets and automatically allocates students based on the available laboratory capacity. The project reduces manual effort, minimizes allocation errors, and provides an organized and user-friendly allocation process. The major objectives of automating the allocation, handling multiple laboratories, and ensuring that laboratory capacity is not exceeded were successfully achieved.

Future Scope

The future development of the project can be carried out in different stages:

Near-Term Enhancements:

- Add export options for matrix tables and optimization results.
- Improve visualization controls and add clearer algorithm explanations.
- Add robust validation for image dimensions and malformed CSV data.

Medium-Term Enhancements:

- Add larger matrix-chain examples and more detailed split-point animations.
- Allow users to configure more algorithm parameters and comparison methods.
- Add optional storage for previously uploaded datasets and results.
- Add downloadable reports for matrix dimensions, costs, and algorithm steps.

Long-Term Enhancements:

- Extend the tool with additional graph and dynamic-programming algorithms.
- Support larger inputs with scalable sampling and visualization.
- Provide API integration for external data sources.
- Add performance comparisons and analytics for different input sizes.

9. REFERENCES

1. **UniTime – University Timetabling System**
   https://www.unitime.org/
2. **UniTime – Student Scheduling**
   https://help-unitime.unitime.org/uct_students.php
3. **UniTime – Examination Timetabling**
   https://help40.unitime.org/examination-timetabling
4. **UniTime – Room Details and Capacity**
   https://help45.unitime.org/room-detail
5. **OptaPlanner – School Timetabling**
   https://docs.optaplanner.org/latest/optaplanner-docs/html_single/
6. **OptaPlanner – Examination Timetabling**
   https://docs.optaplanner.org/6.4.0.Final/optaplanner-docs/html/ch03.html
7. **FET – Free Timetabling Software**
   https://lalescu.ro/liviu/fet/
8. **Springer – University Course Timetabling Research**
   https://link.springer.com/article/10.1007/s10951-021-00715-5
9. **Springer – Curriculum-Based University Course Timetabling**
   https://link.springer.com/article/10.1007/s10100-024-00923-2

10. UniTime Documentation
https://help.unitime.org/

10. ALGORITHMS

This section documents the four algorithms included in the updated project description. The descriptions and standard complexity bounds are provided for reference; their inclusion in this report does not by itself establish that each algorithm is implemented in the application.

10.1 Knuth–Morris–Pratt (KMP)

KMP searches for a pattern in a text using a Longest Prefix Suffix (LPS) table to avoid repeated comparisons. Time complexity: O(n + m), where n is the text length and m is the pattern length.

10.2 Rabin–Karp

Rabin–Karp uses hash values and a rolling hash to identify possible pattern matches. Average time complexity: O(n + m); worst-case time complexity: O(nm), where n is the text length and m is the pattern length.

10.3 Matrix Chain Multiplication (MCM)

MCM uses interval dynamic programming to find the parenthesization that minimizes scalar multiplications. Its recurrence is: dp[i][j] = min over i ≤ k < j of {dp[i][k] + dp[k+1][j] + p[i−1] × p[k] × p[j]}. Time complexity: $O(n^3)$; space complexity: $O(n^2)$.

10.4 Edmonds–Karp

Edmonds–Karp computes maximum flow in a network by using Breadth-First Search (BFS) to find augmenting paths in the residual graph. Time complexity: $O(VE^2)$, where V is the number of vertices and E is the number of edges.